

网络空间安全
理论、技术、应用

王　勇
电子科技大学计算机学院

Cla.R

电子科技大学



第四章　安全检索技术

4.1 基本概念

4.2 早期安全检索技术

4.3 对称密文检索

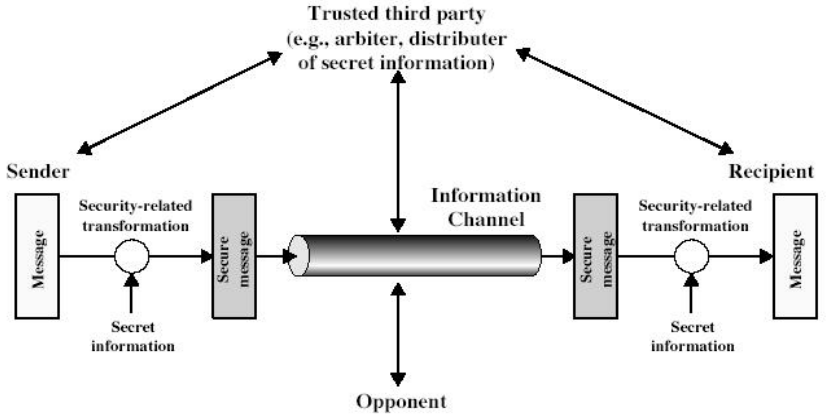
4.4 非对称密文检索

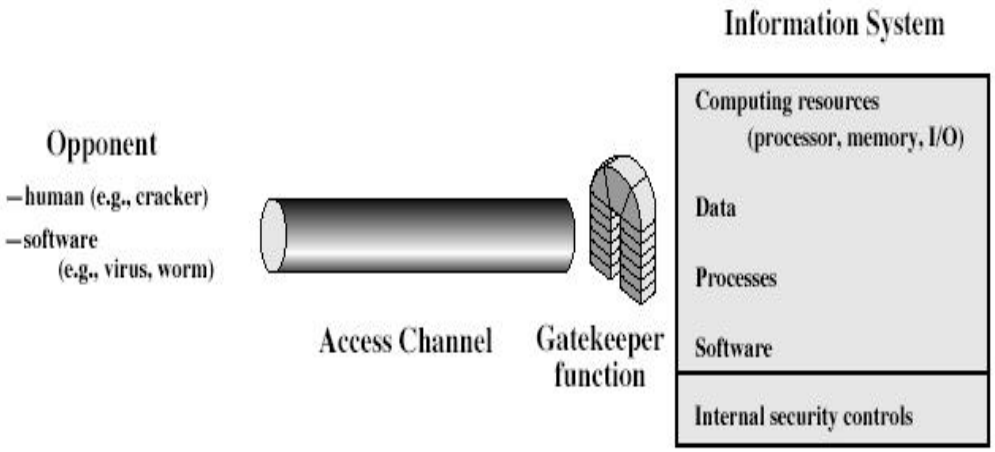
4.5 密文区间检索



两个基本的安全模型

- 网络安全模型
- 访问控制模型







背 景

外在攻击者 Vs 合法的好奇者

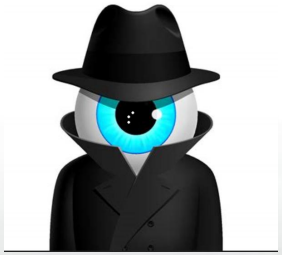


攻击者的知识

- 算法透明
- 信息系统的结构，传输协议，信息使用过程

好奇者的知识

- 算法透明
- 信息系统的结构，传输协议，信息的使用过程
- 信息内容
- 秘密信息



Cla.R

电子科技大学



背 景

外在的攻击者 Vs 合法的好奇者

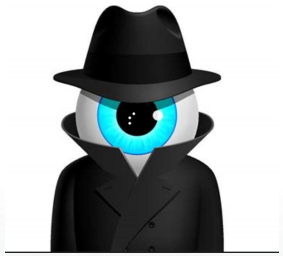


攻击者的行为

- 猜测：brute force attack， dictionar
- 漏洞分析与利用：buffer overflow
- 资源对抗：DoS，DDoS
- 算法分析：DH man-in-the-middle attack

好奇者的行为

- Collect：
- Background knowledge analysis：



Cla.R

电子科技大学



背 景

计算能力、存储能力、传输能力的增加并未能减轻信息处理的成本，因为人们还需要知道更多的信息

社会分工是人类发展的必然过程

云计算、云存储降低了个人（小实体）信息处理的成本

当信息系统和数据迁移到了云端

- 好奇者由内部用户转移为第三方外部实体
- 关于信任问题



Cla.R



Honest-but-curious （HBC） model

the notion of secure multiparty computing . The setting is that there are m parties, each of which has a private input $x_1, \ldots, x_m$. The goal is to compute $f(x_1, \ldots, x_m, R)$ securely, where R is random coins and "securely" means that no party obtains any more knowledge about other parties' private inputs than could be obtained if all computation were done through a trusted third party. The setting for this problem can be thought of as computers on a network; personal computation is private but all interparty communication is up for grabs.

In the HBC setting, every party is obliged to follow the protocol, but cannot intentionally "forget" knowledge that it learns during the execution of the protocol. In other words, all parties are curious, in that they try to find out as much as possible about the other inputs despite following the protocol. A protocol is secure in the HBC sense if and only if all parties have no new knowledge at the end of the protocol above what they would have learned from the output of f.

Cla.R



密文检索

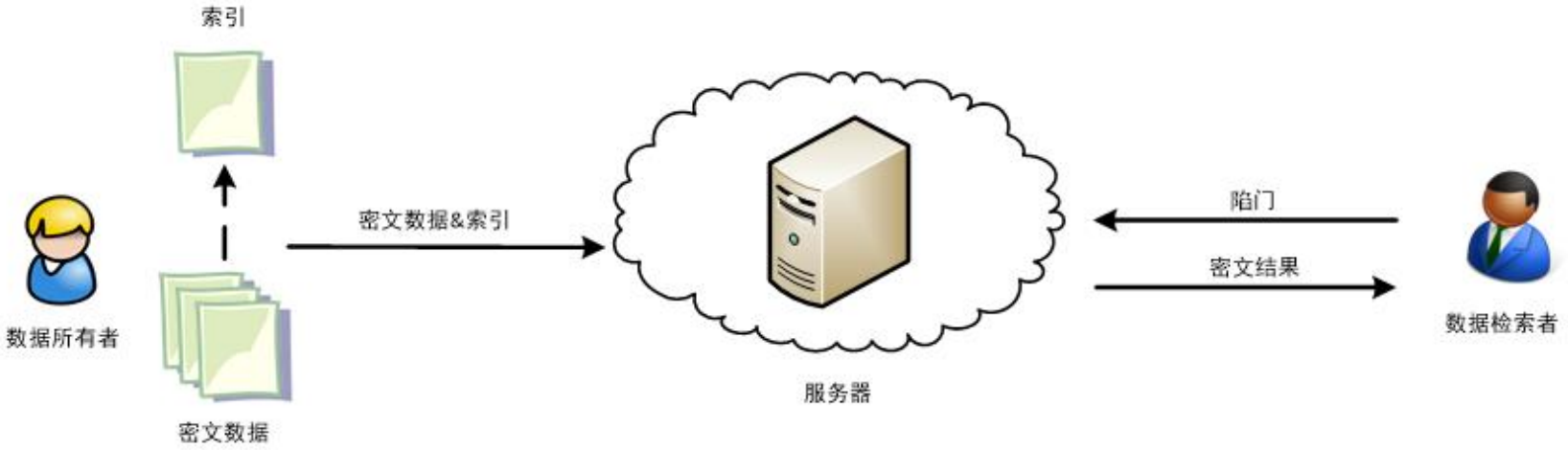
为保证云数据的安全性，一种通用的方法是用户首先使用安全的加密机制（如DES、AES、RSA等）对数据进行加密，然后再将密文数据上传至云服务器。当用户需要对数据进行检索时，只能把全部密文下载到本地，将其解密后再查询。这个过程要求客户端具有较大的存储空间以及较强的计算能力，且没有充分发挥云存储的优势。

密文检索（**Searchable Encryption，SE**）技术支持云存储系统在密文场景下对用户数据进行检索，然后将满足检索条件的密文数据返回给用户。在检索过程中，云服务器无法获得用户的敏感数据和查询条件，即密文检索可以同时保护数据机密性以及查询机密性。



密文检索系统模型

密文检索主要涉及数据所有者、数据检索者以及服务器3种角色。其中，数据所有者是敏感数据的拥有者，数据检索者是查询请求的发起者，这二者通常仅具备有限的存储空间和计算能力；服务器为所有者和检索者提供数据存储和数据查询服务，其由云存储服务提供商进行管理和维护，并具有强大的存储能力和计算能力。





系统工作流程

① 数据所有者首先为数据构造支持检索功能的索引，同时使用传统的加密技术加密全部数据，然后将密文数据和索引共同存储到服务器。

② 需要检索时，数据检索者为检索条件生成相应的陷门，并发送给服务器。

③ 服务器使用索引和陷门进行协议预设的运算，并将满足检索条件的密文数据返回给数据检索者。

④ 数据检索者使用密钥将检索结果解密，得到明文数据。有时服务器返回的密文数据中可能包含不满足检索条件的冗余数据，此时数据检索者还需要对解密后的明文数据进行二次检索，即在本地剔除冗余数据。



密文检索的分类

根据应用场景的不同，密文检索技术可以分为对称密文检索SSE和非对称密文检索ASE两大类：

- 对称密文检索：在对称密钥环境下，只有数据所有者拥有密钥，也只有数据所有者可以提交敏感数据、生成陷门，即数据所有者和数据检索者为同一人。
- 非对称密文检索：在非对称密钥环境下，任何可以获得数据检索者公钥的用户都可以提交敏感数据，但只有拥有私钥的数据检索者可以生成陷门。

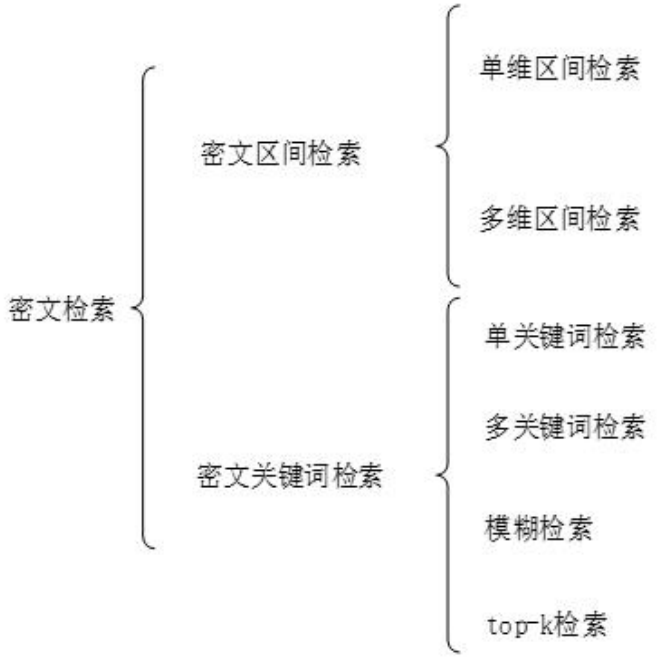
表3-1 对称密文检索和非对称密文检索的比较

特性	对称密文检索	非对称密文检索
密文和索引的构建	由私钥生成	由公开参数生成
密钥管理	单用户场景	多用户场景
性能	高效	低效
解决的问题	不可信服务器存储	不可信服务器路由



根据检索的数据类型的不同，密文检索技术可以分为关键词检索和区间检索两大类：

- 关键词检索：主要用于检索字符型数据，如查询包含关键词“云存储”的文档。最初，关键词检索的研究以单关键词检索为主，后来逐渐扩展到多关键词检索、模糊检索以及Top-k检索。
- 区间检索：主要用于对数值型数据进行范围查询，如查询学生信息表中年龄属性小于18的学生。根据属性的数目，区间检索又可进一步分为单维区间检索和多维区间检索。





早期的安全检索技术

PIR技术的研究主要针对公开数据库，其目标是允许用户在不向服务器暴露查询意图的前提下，对服务器中的数据进行查询并取得指定内容。

根据服务器的数目以及用户与服务器之间的交互轮数的不同，可将PIR技术分为4大类：单服务器的、多服务器的、单轮交互的以及多轮交互的。目前主要研究的是单轮交互的PIR问题。



单轮交互PIR

定义3-1（单轮交互的PIR问题）

设存在$k \geq 1$个服务器，其存储的内容完全相同，均为n个比特的信息$X = \{x_1, x_2, \ldots, x_n\}$，且服务器之间不会进行相互通信。A希望对服务器中的数据进行查询，并得到x_i，其具体查询过程如下：

1. A生成一个随机数r，并根据r和i生成k个查询$\{q_1, q_2, \ldots, q_k\}$，然后将其分别发送给$k$个服务器；
2. 各服务器分别返回相应的查询结果：$\{Ans(q_1), \ldots, Ans(q_k)\}$；
3. A根据r和$\{Ans(q_1), \ldots, Ans(q_k)\}$计算得到正确的$x_i$。

如果在上述查询过程中，任意服务器均不了解关于i的任何信息，则称这一交互是PIR的。换句话说，如果A进行了两次查询，分别访问了x_i和$x_{i'}$，则服务器j对这两次查询所见的视图在概率分布上没有区别，即

$$\forall i, i', j: \Pr\left[View_j(X, i) = view\right] = \Pr\left[View_j(X, i') = view\right]$$



示例方案-1

以k=4为例

(1)将数据X={x1,x2,…,xn}表示为sqrt（n）*sqrt（n）的举证，待查询数据xi表示为xi1，xi2（i1，i2为矩阵中的坐标），四个服务器分别为DB00，DB01，DB10，DB11

（2）A 生成两个长度为$\sqrt{n}$的随机数δ、τ，并计算 $\delta_1=\delta\oplus i_1$，$\tau_1=\tau\oplus i_2$。

（3）A 将δ和τ发送给 DB_{00}，将δ和τ_1发送给 DB_{01}，将δ_1和τ发送给 DB_{10}，将δ_1和τ发送给 DB_{11}。

（4）服务器 DB_{00}返回结果$\oplus_{\delta(j_1)=1,\tau(j_2)=1}x_{j_1,j_2}$，服务器 DB_{01}返回结果$\oplus_{\delta(j_1)=1,\tau_1(j_2)=1}x_{j_1,j_2}$服务器 DB_{10}返回结果$\oplus_{\delta_1(j_1)=1,\tau(j_2)=1}x_{j_1,j_2}$，服务器 DB_{11}返回结果$\oplus_{\delta_1(j_1)=1,\tau_1(j_2)=1}x_{j_1,j_2}$。

（5）A 将返回结果进行异或，得到 x_{i_1,i_2}。

现在考虑上述方案的正确性。由于 x_{i_1,i_2}满足 $\delta(i_1)\neq\delta_1(i_1)$且 $\tau(i_2)\neq\tau_1(i_2)$，因此x_{i_1,i_2}必然在服务器的返回结果中出现 1 次。对于其他任意的 x_{i_3,i_4}，要么 $\delta(i_3)=\delta_1(i_3)$，要么 $\tau(i_4)=\tau_1(i_4)$，因此，它们都会出现偶数次，会在异或过程中被消除。



示例方案-2

定义 3-2(二次剩余问题) 设 z 和 m 是两个互素的整数,如果存在整数 α 使得 $\alpha^2 =$ od m,则称 z 是一个在模 m 运算上的二次剩余。

一般认为,给定 z 和 m,如果 m 是两个大素数的积,且敌手不了解 m 的因子分解,则敌定 z 是否是 QR 是困难的(与对 m 进行因子分解的难度相当)。

(1) 记 $k=\sqrt{n}$,将大小为 n 的数据库 X 表示为一个 $k\times k$ 的矩阵,则待查询的数据 x_i 可示为 $x_{a,b}|a,b\leqslant k$。

(2) A 生成两个二进制等长的素数 p_1、p_2,$N=p_1p_2$。

(3) A 从 $\mathbf{Z}_N^*$ 中选择 k 个雅可比符号为 1 的元素 $y_1,y_2,\cdots,y_k$,其中除 y_b 之外均QR。

(4) A 将 N 以及 $y_1,y_2,\cdots,y_k$ 发送给服务器。

(5) 服务器对所有元素 $x_{i,j}$ 计算 $C_{i,j}$:如果 $x_{i,j}=0$,则 $C_{i,j}=y_j^2$;否则,$C_{i,j}=y_j$。

(6) 服务器返回 $\left\{r_1=\prod_{j=1}^{k}C_{1,j},r_2=\prod_{j=1}^{k}C_{2,j},\cdots,r_k=\prod_{j=1}^{k}C_{k,j}\right\}$。

(7) A 进行观察,如果 r_a 是 QR,则 $x_{a,b}=0$,否则,$x_{a,b}=1$(因为 A 了解 N 的因子分解,以他可以判定 r_a 是否是 QR)。

现在考虑上述 PIR 方案的正确性。若 $x_{a,b}=0$,则 r_a 必然是一组 QR 的乘积,依然是;否则,r_a 是一组 QR 乘以 y_b,必然不是 QR。



单轮交互PIRK

PIR方案假设数据是二进制的，且客户端已经了解待获取的数据在数据集中的位置。但是在实际检索场景中，客户端一般都是输入一个感兴趣的关键词，然后服务器根据该关键词找到对应的数据内容。为此，提出了PIRK（Private Information Retrieval by Keywords）技术。

定义4-2 （PIRK问题）

设存在k个服务器，其存储的内容完全相同，均为n个长度为l的字符串$S = \{s_1, s_2, \dots, s_n\}$，且服务器之间不会进行相互通信。A感兴趣的关键词是一个长度为l的字符串w。

如果存在一个协议使A能够得到所有满足$s_j = w$的j，且任意服务器均不了解关于w的任何信息，则称该协议是$PIRK(l, n, k)$的。

需要注意的是，此定义只包含了找到$s_j = w$的过程，至于找到s_j之后如何获取s_j对应的数据内容，则可以通过运行一般的PIR协议来完成。



SPIR技术

从服务器的角度来看，PIR技术仅保护了客户端的查询意图，而对服务器中的数据集缺乏保护。因此，由PIR技术进一步发展至SPIR（Symmetric Private Information Retrieval）技术，其目标是将保护范围扩大到服务器。

SPIR问题与PIR问题相似，但是在其基础上增加了一项要求：A不了解x_i之外的任何信息。换句话说，如果存在两个数据源$X = \{x_1, x_2, \ldots, x_n\}$和 $Y = \{y_1, y_2, \ldots, y_n\}$，且$x_i = y_i$。则对于这两个数据源，A查询第$i$份数据，所见的视图应当没有任何区别，即

$$\Pr[View_A(X, i) = view] = \Pr[View_A(Y, i) = view]$$



ORAM技术

ORAM技术是面向秘密数据库的，其目标是在读写过程中向服务器隐藏用户的访问模式。这里，访问模式是指客户端向服务器发起访问所泄露的信息，包括操作是读还是写、操作的数据地址、操作的数据内容等。

与PIR的不同之处在于，PIR只考虑保护客户端的查询意图，整个数据库的内容对服务器是可见的；而ORAM则认为整个服务器的存储介质都是不安全的，因此要求数据是加密的，同时向服务器隐藏读写两种操作。



ORAM系统

定义4-3（ORAM系统）

- 用户的输入序列Y：定义为一组输入$(o_1, o_2, \dots, o_n)$。
- 用户的输入o：代表用户的操作类型、操作数据内容、操作地址$o = (op, data, i)$。其中当op是读时$op = read, data = \emptyset$；当$op$是写时$op = write$，$data$是用户写入的明文内容。
- 访问模式$A(Y)$：对于用户发起的一个输入序列$Y = (o_1, o_2, \dots, o_n)$，假设经过翻译后，在服务器实际实施的序列为$Y = (O_1, O_2, \dots, O_m)$。$A(Y)$记录了$Y$中各输入的访问地址$i$，以及$Y$是读还是写。当操作是写时，$A(Y)$还记录了用户希望服务器写入的内容。具体地，$A(Y) = ((op_1, Edata_1, i_1), \dots, (op_m, Edata_m, i_m))$，其中$Edata$是从服务器的角度写入的密文内容。当操作是读时，$Edata = \emptyset$。

如果对于系统中任意两个输入序列Y和Y'，从服务器的角度来看，访问模式$A(Y)$和$A(Y')$是不可区分的，则认为这个系统是一个ORAM系统。



4.3 对称密文检索

在对称密文检索方案中，数据所有者和数据检索者为同一方。该场景适用于大部分第三方存储，也是近几年本领域的研究热点。一个典型的对称密文检索方案包括如下算法：

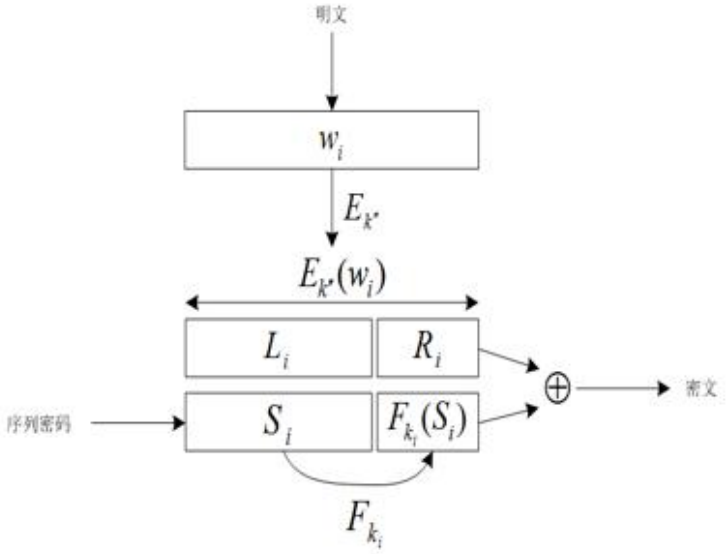
- Setup算法：该算法由数据所有者执行，生成用于加密数据和索引的密钥；
- BuildIndex算法：该算法由数据所有者执行，根据数据内容建立索引，并将加密后的索引和数据本身上传到服务器；
- GenTrapdoor算法：该算法由数据所有者执行，根据检索条件生成相应的陷门（又称“搜索凭证”），然后将其发送给服务器；
- Search算法：该算法由服务器执行，将接收到的陷门和本地存储的密文索引作为输入，并进行协议所预设的计算，最后输出满足条件的密文结果。

目前，SSE可根据检索机制的不同大致分为三大类：基于全文扫描的方法、基于文档-关键词索引的方法以及基于关键词-文档索引的方法。



基于全文扫描的方案

最早的对称密文检索方案由Song等人提出，其是一种基于全文扫描的方案。该方案的核心思想是对文档进行分组加密，然后将分组加密结果与一个伪随机流进行异或得到最终用于检索的密文。检索时，用户将检索关键词对应的陷门发送给服务器，服务器对所有密文依次使用陷门计算密文是否满足预设的条件，若满足则返回该文档。





方案的具体步骤概述如下，该方案构造的密文和陷门与伪随机数具有不可区分性：

- Setup算法：数据所有者生成密钥k', k''，伪随机数$S_1, S_2, \ldots, S_l$，伪随机置换E以及伪随机函数F, f；
- BuildIndex算法：假设文档的内容为关键词序列$w_1, w_2, \ldots, w_l$。对于关键词w_i，数据所有者首先将其加密得到$E_{k''}(w_i)$，并将$E_{k''}(w_i)$拆分为L_i和R_i两个部分；然后，使用伪随机数S_i计算$F_{k_i}(S_i)$，其中$k_i = f_{k'}(L_i)$；最后，将$(S_i, F_{k_i}(S_i))$与(L_i, R_i)经过异或运算生成密文块C_i；
- GenTrapdoor算法：当需要搜索关键词w时，数据所有者将$E_{k''}(w) = (L, R)$以及$k = f_{k'}(L)$发送给服务器；
- Search算法：服务器依次将密文C_i与$E_{k''}(w)$进行异或运算，然后判断得到的结果是否满足$(S, F_k(S))$的形式。如果满足，则说明匹配成功，并将该文档返回。



效率和安全问题

基于全文扫描的方案需要对每个密文块进行扫描并计算，在最坏的情况下，检索一篇文档的时间与该文档的长度呈线性关系，检索效率较低。

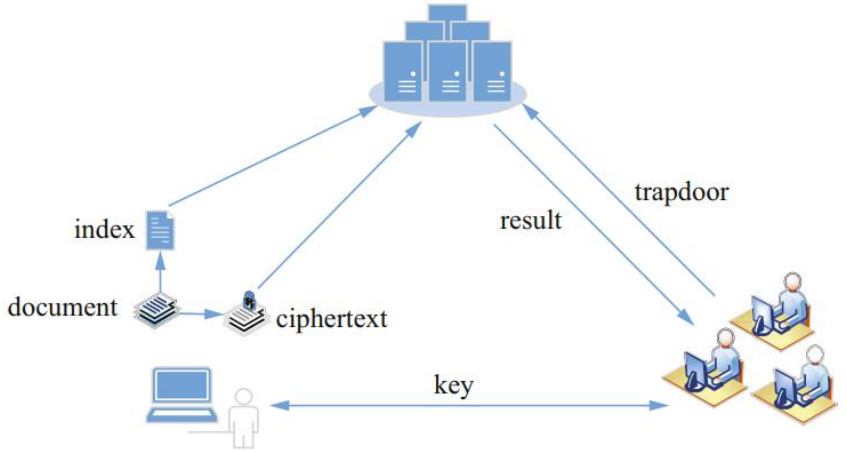


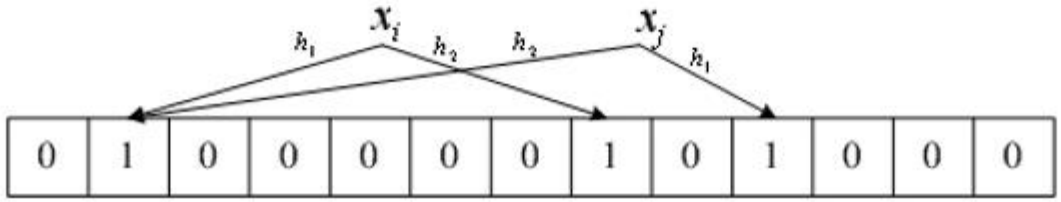
Figure 1 Model of SSE schemes



基于文档-关键词索引的方案

布隆过滤器利用位数组表示集合，并可以快速判断一个元素是否属于该集合。记位数组的长度为m，集合为$S = \{x_1, \dots, x_n\}$。首先，构造各位置均为0的初始数组BF，并选取k个Hash函数$h_1, \dots, h_k$，这些Hash函数可以将集合中的元素映射到位数组中的某一位。然后对于各元素x_i，为其计算k个Hash值$h_1(x_i), \dots, h_k(x_i)$，并将位数组中的相应位置设为1。

当想要判断元素y是否属于集合S时，我们同样使用Hash函数$h_1, \dots, h_k$为其计算k个值$h_1(y), \dots, h_k(y)$，如果位数组BF中的相应位置均为1，我们则认为y是S中的元素。但实际上，由于Hash函数的计算结果可能存在冲突，y有可能并不属于S。如图3-4所示的例子，如果$h_1(y) = h_2(x_i)$且$h_2(y) = h_1(x_j)$，则会发生误判。





基于布隆过滤器的密文关键词检索方案

基于布隆过滤器的密文关键词检索方案ε：使用布隆过滤器为每篇文档分别构造索引，并使用伪随机函数为每个关键词计算两遍伪随机数，其一将关键词作为输入，其二将文档标识作为输入，从而同一关键词在不同文档中的计算结果不一致。具体方案概述如下：

- Setup算法：数据所有者生成r个密钥$k_1, \dots, k_r$以及伪随机函数f；
- BuildIndex算法：对于包含t个关键词$w_1, \dots, w_t$的文档D，数据所有者首先为其生成一个位数组BF(D)，并置BF(D)所有位均为0。然后，对于每个关键词w_i：
  - 以关键词w_i作为输入计算r个值：$x_1 = f(w_i, k_1), \dots, x_r = f(w_i, k_r)$；
  - 以文档标识id作为输入计算r个值：$y_1 = f(id, x_1), \dots, y_r = f(id, x_r)$；
  - 将BF(D)中$y_1, \dots, y_r$这r个值对应的位置设为1，并对BF(D)进行随机填充；
- GenTrapdoor算法：数据所有者为检索关键词w计算r个值：$x'_1 = f(w, k_1), \dots, x'_r = f(w, k_r)$，然后将这$r$个值发送给服务器；
- Search算法：根据陷门，服务器为文档D计算r个值$y'_1 = f(id, x'_1), \dots, y'_r = f(id, x'_r)$，并检查D对应的索引BF(D)中，这$r$个值对应的位置是否都为1。若是，则说明文档D包含$w$，并将其返回给用户。



选择关键词语义安全IND2-CKA的含义是：对于两个数据文档V_0和V_1，仅凭其密文索引无法对二者进行区分。为了定义IND2-CKA，我们先定义如下游戏：

- Setup过程：挑战者C创建一个关键词集合S，并将其发送给敌手A（可将敌手A视作一个概率多项式时间（PPT）算法。A选择S的若干个子集，这些子集的集合记为S*，并将S*返回给C。此处，一个子集可以看成一个数据文档。C运行Setup算法，并对S*的每个元素运行BuildIndex算法，最后将全部索引及其对应的子集发送给A；
- Query过程：敌手A向挑战者C请求关键词x的陷门T_x，并在任意索引上运行Search算法以判定该索引是否包含x；
- Challenge过程：在运行若干次Query之后，敌手A从S*中选择两个非空子集V_0和V_1，且$|(V_0 - V_1) \cup (V_1 - V_0)| \neq 0$，$(V_0 - V_1) \cup (V_1 - V_0)$中的任意关键词均未被查询过。A将$V_0$和$V_1$发送给C，C随机抛掷硬币$b$，并在$V_b$上运行BuildIndex算法，最后将对应的结果发给A；
- Response过程：敌手A给出对b的猜测b'。

我们将上述游戏的优势定义为$Adv_A = |\Pr(b' = b) - 1/2|$。



较为常见的安全模型

- **IND-CKA（indistinguishability under chosen keyword attack）模型**

- **IND-KPA（indistinguishability under chosen-keyword plaintext attack）模型**

- **IND-KIA（indistinguishability under chosen-keyword identity attack）模型**



关键词检索方案示例-3

势都是可忽略的。

定理 3-1 如果函数 f 是一个伪随机函数，则方案 ε_1 满足 IND2-CKA 安全性。

通过使用掩码技术，人们实现了一种误报率为 0 的密文关键词检索方案 ε_2。方案 ε_2 核心思路是：提前为关键词集构造字典，并由用户将其保存在本地。字典包含 2^d 对 $(i, w_i$ 其中 $w_i \in \{0,1\}^*$ 代表一个关键词，$i \in [1, 2^d]$ 为 w_i 对应的唯一值。具体方案概述如下：

(1) Setup 算法。数据所有者生成密钥 s、r，伪随机置换函数 P 以及伪随机函数 F、G。

(2) BuildIndex 算法。对于文档 D，数据所有者首先生成一个长度为 2^d 的初始位串 I 其各位均为 0。如果文档 D 包含关键词 w_i，则将位串 I 的第 $P_s(i)$ 位设为 1，即 $I[P_s(i)]=1$ 然后，将 I 的各位 $I[j]$ 与其对应的掩码 $G_{F_r(j)}(\mathrm{id})$ 进行异或操作，即 $I[j]=I[j]\oplus G_{F_r(j)}(\mathrm{id})$。最后，将位串 I 发送给服务器作为索引。

(3) GenTrapdoor 算法。数据所有者为检索关键词 w_λ 生成 $p=P_s(\lambda)$ 和 $f=F_r(p)$，并将其发送给服务器。

(4) Search 算法。服务器计算 $I[p]\oplus G_f(\mathrm{id})$，如果结果为 1，则将文档 D 返回给数据所



模拟安全性

定义**4-4**（模拟安全性）

假设C_q为服务器在前q轮查询中获得的信息，则对于任意概率多项式时间算法A，任意集合$H=\{D_1,\ldots,D_n,w_1,\ldots,w_q\}$以及任意函数$f$，均存在一个概率多项式时间算法S，使得

$$|\Pr[A(C_q)=f(H)]-\Pr[S(\{\varepsilon(D_1),\ldots,\varepsilon(D_n),D(w_1),\ldots,D(w_q)\})=f(H)]|\leq negl(k)$$

其中，$\varepsilon(D_i)$是文档D_i的加密结果，$D(w_j)$是包含关键词w_j的文档集，k是安全参数。



基于关键词-文档索引的方案

在基于文档-关键词索引的方案中，查询效率与文档数目呈线性关系，导致这类方案难以应用于大数据场景。为此，提出了基于关键词-文档索引的方案。此类方案的索引结构类似于搜索引擎倒排索引，在初始化时为每个关键词生成包含该关键词的文档标识集合，然后加密存储这些索引结构。基于关键词-文档索引的方案不需要逐个检索每篇文档，其检索时间复杂度仅与返回的结果数目呈线性关系，因此查询效率远高于前两类方案。



倒排文件

Inverted file contains a list of pointers (e.g. the number of a page) to all occurrences of that term in the text.

Inverted because it lists for a term, all documents that contain the term

Doc	Text
1	Gold silver truck
2	Shipment of gold damaged in a fire
3	Delivery of silver arrived in a silver truck
4	Shipment of gold arrived in a truck

Inverted File Index

No.	Term	Times; Documents
1	a	<3; 2,3,4>
2	arrived	<2; 3,4>
3	damaged	<1; 2>
4	delivery	<1; 3>
5	fire	<1; 2>
6	gold	<3; 1,2,4>
7	of	<3; 2,3,4>
8	in	<3; 2,3,4>
9	shipment	<2; 2,4>
10	silver	<2; 1,3>
11	truck	<3; 1,3,4>



基于关键词-文档索引的方案

基于关键词-文档索引的方案π的索引包括一个数组A以及一个查找表T。其核心思路是首先为包含关键词w_i的第j篇文档构造节点$N_{i,j}$，此节点包含该文档的标识、下一个节点的加密密钥及其在数组A中的存储位置，然后将此节点加密保存在数组A中。最后，将第一个节点$N_{i,1}$的加密密钥以及存储位置异或一个掩码后存储在查找表T中。检索时，服务器首先通过查找表T找到检索关键词对应的第一个节点的信息，然后对数组A进行查找和解密，直到获得检索关键词对应的最后一个节点。



方案步骤

- Setup算法：数据所有者生成密钥K_1, K_2, K_3，对称加密算法Enc，伪随机置换φ, π以及伪随机函数f；
- BuildIndex算法：给定文档集合D，数据所有者首先对其进行分词得到关键词集合$\delta(\mathrm{D})$，并为每个关键词$w_i \in \delta(\mathrm{D})$生成所有包含该关键词的文档集合$\mathrm{D}(w_i)$。然后，初始化计数器ctr=1、数组A以及查找表T。对于各关键词w_i，$1 \le i \le |\delta(\mathrm{D})|$：
  - 生成密钥$K_{i,0}$；
  - 对于$1 \le j \le |\mathrm{D}(w_i)| - 1$：首先生成密钥$K_{i,j}$，然后构造节点$N_{i,j} =< id(D_{i,j})||K_{i,j}||\varphi_{K_1}(\mathrm{ctr}+1) >$，其中$id(D_{i,j})$为$\mathrm{D}(w_i)$中第$j$篇文档的标识。最后，使用密钥$K_{i,j-1}$加密节点$N_{i,j}$，并将其保存在数组A的第$\varphi_{K_1}(\mathrm{ctr})$个位置，即$A[\varphi_{K_1}(\mathrm{ctr})] = Enc_{K_{i,j-1}}(N_{i,j})$，同时令ctr=ctr+1；
  - 对于$j = |\mathrm{D}(w_i)|$：构造节点$N_{i,|\mathrm{D}(w_i)|} =< id(D_{i,|\mathrm{D}(w_i)|})||0^k||NULL >$，使用密钥$K_{i,|\mathrm{D}(w_i)|-1}$加密该节点，并将其保存在数组A的第$\varphi_{K_1}(\mathrm{ctr})$个位置，即$A[\varphi_{K_1}(\mathrm{ctr})] = Enc_{K_{i,|\mathrm{D}(w_i)|-1}}(N_{i,|\mathrm{D}(w_i)|})$，同时令ctr=ctr+1；
  - 置$T[\pi_{K_3}(w_i)] =< addr_A(N_{i,1})||K_{i,0} > \oplus f_{K_2}(w_i)$，其中$addr_A(N_{i,1})$为节点$N_{i,1}$在数组A中的存储位置。最后，对数组A以及查找表T中剩下的0进行混淆；
- GenTrapdoor算法：数据所有者为检索关键词w生成陷门$t = (\pi_{K_3}(w), f_{K_2}(w))$；

Search算法：服务器首先根据陷门$t = (\gamma, \eta)$得到$\theta = T[\gamma]$。如果$\theta \neq \perp$，则通过计算$\theta \oplus \eta$得到$<\alpha||K'>$，然后使用K'解密$A[\alpha]$保存的节点，并得到该节点对应的文档标识以及后续节点在数组A中的存储位置和解密密钥，从而依次获得其后续节点的内容。最后服务器将检索到的文档标识返回给数据所有者。



IND-CKA安全性

为阐述**IND2-CKA安全性的缺陷**，构造如下密文检索方案：

记$\Delta=\{w_1,\dots,w_d\}$为关键词集合，为每篇文档分别构造索引，具体步骤如下：

- Setup算法：数据所有者生成密钥K以及伪随机置换函数π；
- BuildIndex算法：假设数据所有者为文档集合中的第ctr篇文档D构造索引，首先初始化一个长度为d的数组A，然后对于关键词集合中的各关键词$w_i \in \Delta$，如果文档D包含关键词w_i，则执行如下操作：
  - 计算$r=\pi_K(w_i||ctr)$；
  - 将A[i]设置为$r\oplus(w_i||0^k)$，并在数组A中没有存储数据的位置填充随机串；
- GenTrapdoor算法：数据所有者为检索关键词w生成陷门$r_1=\pi_K(w||1),\dots,r_n=\pi_K(w||n)$，其中n为文档集合中的文档总数，即每篇文档对应的陷门不同；
- Search算法：对于文档D对应的数组A，如果存在$1\le j\le|A|$，使得$A[j]\oplus r_{ctr}$的后k位均为0，则输出1；否则，输出0。

定理3-2 如果π是伪随机置换函数，则上述方案满足IND2-CKA安全性。

显然，虽然上述方案可以满足IND2-CKA安全性，但是由于Search算法直接暴露了关键词w的明文内容，该方案实际上是不安全的。究其根本原因，在于IND2-CKA安全性只考虑了索引和陷门孤立的安全性，而检索操作是需要索引和陷门二者同时作为输入的，还需要考虑二者结合的安全性。



安全性定义

为此提出了4个新的安全性定义：非适应性语义安全NS、非适应性不可区分性NI、适应性语义安全AS、适应性不可区分性AI，它们的安全级别关系是NS=NI<AI<=AS。在描述具体的安全性定义之前，首先介绍几个辅助概念：

定义4-4（查询历史）一个查询历史H包括两个组成部分：被查询的文档集合D=$\{D_1, \dots, D_n\}$以及查询关键词列表W=$\{w_1, \dots, w_q\}$，即H=(D,W)。

定义4-5（访问模式）一个查询历史H的访问模式α是所有查询返回的文档列表：$\alpha(H) = (D(w_1), \dots, D(w_q))$。

定义4-6（搜索模式）一个查询历史H的搜索模式δ是一个对称的q×q矩阵，表示两次查询的关键词是否相等：当且仅当$w_i = w_j$时，矩阵δ(H)第i行j列的元素为1即$\delta(H)[i,j] = 1$；否则，$\delta(H)[i,j] = 0$。

定义4-7（轨迹）一个查询历史H的轨迹t包括文档集合中每篇文档的长度以及访问模式和搜索模式：$t(H) = (|D_1|, \dots, |D_n|, \alpha(H), \delta(H))$。



非适应性语义安全

设Real是如下的一个游戏过程：

1. 挑战者运行Setup算法获得密钥；
2. 敌手选择查询历史H=(D,W)；
3. 挑战者对文档集合D加密得到密文[D]，并运行BuildIndex算法得到索引I。同时，运行GenTrapdoor算法为查询关键词列表W中的各关键词w_i生成陷门t_i；
4. 输出$(I, [D], t_1, \dots, t_q)$。

设Sim是如下的一个模拟过程：

1. 敌手选择查询历史H=(D,W)；
2. 挑战者根据轨迹t(H)模拟生成$(I, [D], t_1, \dots, t_q)$；
3. 输出$(I, [D], t_1, \dots, t_q)$。

称一个密文检索方案是非适应性语义安全的，如果对于任意多项式能力的敌手，均存在一个多项式时间模拟算法，使得Real和Sim的输出结果无法区分。



适应性语义安全

Real是一个如下的游戏过程：

1. 挑战者运行Setup算法得到密钥；

2. 敌手选择文档集合D；

3. 挑战者对文档集合D加密得到密文[D]，并运行BuildIndex算法得到索引I；

4. 敌手根据密文[D]和索引I选择第一个查询关键词w_1；

5. 挑战者运行GenTrapdoor算法生成关键词w_1对应的陷门t_1；

6. 对于$2 \le i \le q$：

6.1）敌手根据[D]、I以及前i-1次的陷门$t_1,\ldots,t_{i-1}$选择查询关键词w_i；

6.2）挑战者运行GenTrapdoor算法生成关键词w_i对应的陷门t_i；

7. 输出$(I, [D], t_1, \ldots, t_q)$。

的



适应性语义安全

Sim是一个如下的模拟过程：

1. 敌手选择文档集合D；

2. 挑战者根据轨迹t(D)模拟生成密文[D]以及索引I；

3. 敌手根据密文[D]和索引I选择第一个查询关键词w_1；

4. 挑战者根据轨迹t(D,w_1)模拟生成陷门t_1；

5. 对于2≤i≤ q：

5.1）敌手根据[D]、I以及前i-1次的陷门$t_1,\ldots, t_{i-1}$选择查询关键词w_i；

5.2）挑战者根据轨迹t(D,$w_1,\ldots,w_i$)模拟生成陷门t_i；

6. 输出$(I, [D], t_1, \ldots, t_q)$。



适应性语义安全

称一个密文检索方案是适应性语义安全的，如果对于任意多项式能力的敌手，均存在一个多项式时间模拟算法，使得Real和Sim的输出结果无法区分。

扩展1：多关键词SSE检索

在实际应用场景中，用户通常以多个关键词作为检索条件。基于关键词-文档索引的多关键词检索方案δ的核心思想是首先获取一个结果数目较少的子查询，然后根据字典索引过滤该子查询，从而获取最终的查询结果。详细步骤如下：

1. Setup算法：数据所有者生成密钥k_S,k_X,k_I,k_Z，伪随机函数F,H，以及对称加解密算法Enc,Dec;

2. BuildIndex算法：数据所有者首先对文档集合进行分词，然后建立两个空的索引数组XSet和T，其中T[w_i]由关键词w_i索引。对于每个关键词w_i:

2.1）计算$K_e=F(k_S,w_i)$;

2.2）对于包含关键字w_i的每篇文档D_n:

2.2.1）将w_i使用伪随机函数映射到一对值(A_i,B_i):$A_i=H(k_Z,w_i)$, $B_i=H(k_X,w_i)$，并计算文档标识id(D_n)的哈希值以及密文: $X_n=H(k_I, id(D_n))$, $Y_n=Enc(K_e, id(D_n))$；

2.2.2）在表T[w_i]中插入值$(Y_n, X_nA_i^{-1})$，在表XSet中插入值$g^{X_nB_i}$;

3. GenTrapdoor算法：当需要检索同时含有关键词w_1和w_2的文档时，用户将$R=g^{B_2A_1}$发送到服务器;

4. Search算法：服务器首先找到表T中所有对应w_1的$(Y_n, X_nA_1^{-1})$，然后计算$R^{X_nA_1^{-1}}$是否存在于表XSet中。若是，则证明该检索结果同时也含有w_2，将其对应的Y_n返回给用户。



只要注意选取出现频次足够稀少的关键词作为查询”锚点”（即上文中的关键词w_1），方案δ的查询效率是很高的。以双关键字查询$\{w_1,w_2\}$为例，求交集方法的复杂度至少为$O(|D(w_1)|+|D(w_2)|)$（如果文档标识是无序保存的，复杂度还会更高），而方案δ的复杂度为$O(|D(w_1)|)$。

在安全性方面，该方案存在细微的隐私泄露：如果曾经检索过$\{w_1,w_2\}$，而后又检索过$\{w_3,w_2\}$，那么会泄露$D(w_1)\cap D(w_3)$。因为对于$D\in D(w_1)\cap D(w_3)$，两个查询得到的计算结果是相同的。但是相对于本方案对多关键字检索提升的效率而言，我们认为这样的泄露是可接受的。



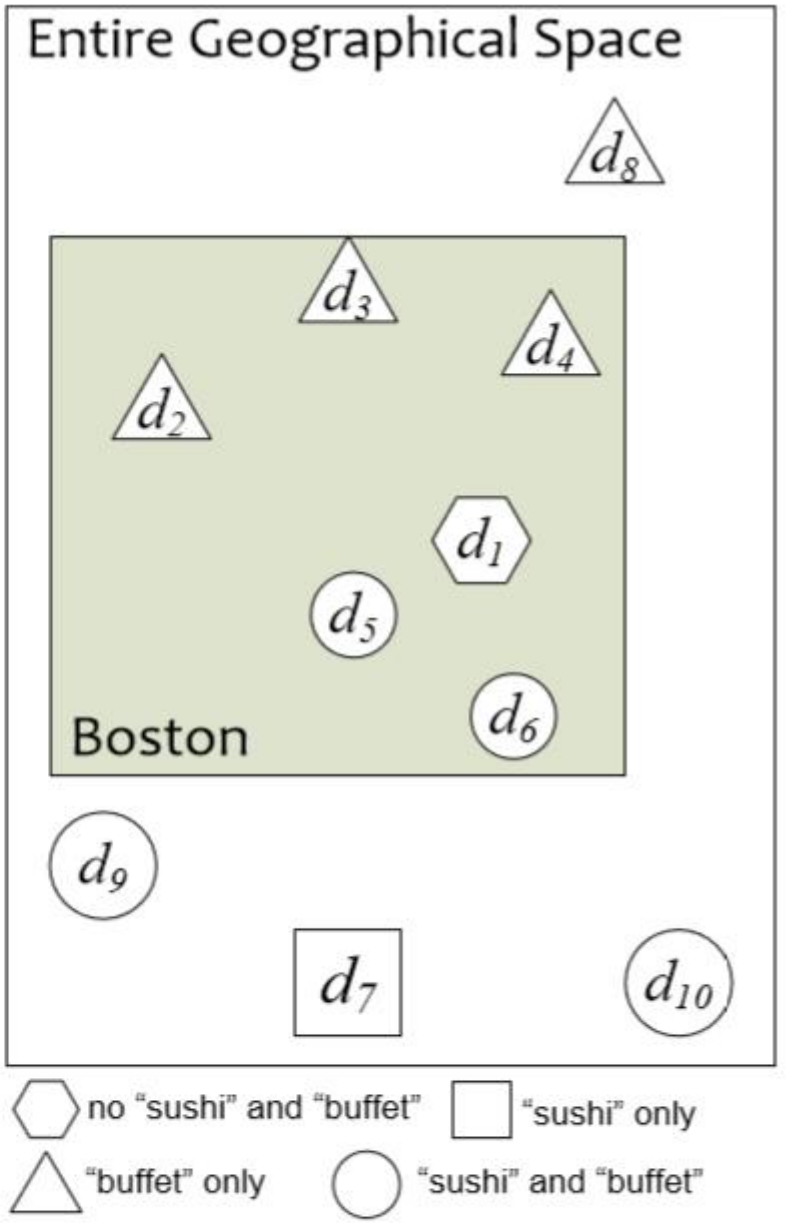
2. Top-k检索

Top-k检索用于对搜索结果进行排序，其与明文的搜索引擎类似，可以按照文档与搜索关键字的相关度给出前k个搜索结果。需要注意的是，按照密文检索的问题背景（即服务器不了解数据的真实内容），要求服务器对检索结果进行排序这一需求本身就造成了隐私泄露。

目前，人们普遍借鉴明文关键词检索中常用的TF-IDF准则来评判文档的相关性。TF-IDF准则认为对一篇文档最有意义的应该是在该文档中出现频率较高，且在整个文档集合中出现频率较低的关键词。

	document words	
	sushi	*buffet*
d_1	0	0
d_2	0	6
d_3	0	8
d_4	0	3
d_5	1	1
d_6	2	1
d_7	1	0
d_8	0	1
d_9	2	2
d_{10}	1	7
df	2	5
idf	$\log \frac{6}{2}$	$\log \frac{6}{5}$

(a) Word and document frequency



(b) Document spatial distribution

Fig. 1. Example documents



前向安全

扩展3：前向安全性扩展

人们感兴趣的另一类安全性是前向安全（Forward Privacy）或称动态安全，是指当系统中新增加一个密文数据时，敌手无法判断该数据是否满足此前的某次查询条件。Zhang等人提出，如果一个密文检索方案不满足前向安全，则只需要插入大约10个新的密文数据即可判断出某次查询对应的关键词明文。换言之，一个能够主动上传指定数据的敌手，可以对任何非前向安全的SSE方案形成破解查询明文攻击。



小结和思考

最初人们提出了选择关键词语义安全IND-CKA和IND2-CKA，未对陷门的安全性提出明确要求

基于此问题，提出了模拟的安全性定义以及4个新的安全性定义（非适应性语义安全，非适应性不可区分性，适应性语义安全，适应性不可区分性）

进一步，基于动态性安全问题，提出了前向、后向安全

思考：沿着这个思路，关键词-文档间的关系受到保护了吗？又该如何保护？

Cla.R



4. 非对称密文检索

非对称密文检索是指数据所有者（数据发送者）和数据检索者（数据接收者）不是同一方的密文检索技术。与非对称密码体制类似，数据所有者可以是了解公钥的任意用户，而只有拥有私钥的用户可以生成检索陷门。一个典型的非对称密文检索过程如下：

- Setup算法：该算法由数据检索者执行，生成公钥PK和私钥SK；
- BuildIndex算法：该算法由数据所有者执行，根据数据内容建立索引，并将公钥加密后的索引和数据本身上传到服务器；
- GenTrapdoor算法：该算法由数据检索者执行，将私钥和检索关键词作为输入，生成相应的陷门（又称“搜索凭证”），然后将陷门发送给服务器；
- Search算法：该算法由服务器执行，将公钥、接收到的陷门和本地存储的索引作为输入，进行协议所预设的计算，最后输出满足条件的搜索结果。



BDOP-PEKS方案

考虑如下应用场景：

邮件发送者B在向邮件接收者A发送邮件时，首先使用A的公钥对邮件包含的各关键词$w_1, w_2, \ldots, w_n$分别构造相应的索引$C_1, C_2, \ldots, C_n$，并将其附在发送的消$E(msg)$后面，一同交由服务器存储。其中E为标准的公钥加密算法，msg为邮件内容。检索时，A使用自己的私钥为查询关键词w生成陷门T_w，并将其发送给服务器，从而服务器能够判断邮件中是否包含关键词w。在这个过程中，服务器无法获得关于邮件内容和查询关键词的任何有用信息。



BDOP-PEKS方案是基于BF-IBE实现的，其安全性可归结为BDH假设。给定两个阶为p的群G_1和G_2，双线性映射$e:G_1\times G_1\rightarrow G_2$，以及两个哈希函数$H_1$和$H_2$，其中$H_1$可以将输入值映射到群$G_1$，BDOP-PEKS方案的具体步骤如下：

- Setup算法：A选择随机数$\alpha = Z_p^*$以及群G_1的生成元g，输出私钥$sk = \alpha$以及公钥$pk = (g, h = g^{\alpha})$；
- BuildIndex算法：对于关键词w_i，B首先选取随机数$r = Z_p^*$，同时计算$t = e(H_1(w_i), h^r) \in G_2$，随后输出$w_i$对应的索引$C_i = (g^r, H_2(t))$；
- GenTrapdoor算法：对于查询关键词w，A使用私钥sk计算相应的陷门$T_w = H_1(w)^{\alpha} \in G_1$；
- Search算法：对于索引C_i =(A,B)，如果$H_2\left(e(T_w, A)\right) = B$，则匹配成功，否则，匹配失败。



KR-PEKS方案

KR-PEKS方案是在KR-IBE的基础上实现的，其安全性可归结为DDH假设.同BDOP-PEKS方案一样，KR-PEKS也是对文档对应的各关键词分别生成索引，并在检索时依次使用陷门测试各索引。方案的具体步骤如下：

- Setup算法：数据检索者选定阶为q的群G，以及两个生成元g_1和g_2，并在Z_q随机构造6个多项式

$$P_1(x)=d_0+d_1x^1+d_2x^2+\ldots+d_kx^k,\quad P_2(x)=d_0^{'}+d_1^{'}x^1+d_2^{'}x^2+\ldots+d_k^{'}x^k,$$

$$F_1(x)=a_0+a_1x^1+a_2x^2+\ldots+a_kx^k,\quad F_2(x)=a_0^{'}+a_1^{'}x^1+a_2^{'}x^2+\ldots+a_k^{'}x^k,$$

$$h_1(x)=b_0+b_1x^1+b_2x^2+\ldots+b_kx^k,\quad h_2(x)=b_0^{'}+b_1^{'}x^1+b_2^{'}x^2+\ldots+b_k^{'}x^k,$$

对于0≤t≤k，计算$A_t=g_1^{a_t}g_2^{a't}$，$B_t=g_1^{b_t}g_2^{b't}$，$D_t=g_1^{d_t}g_2^{d't}$。此外，还需要选择两个随机抗碰撞哈希函数H和H′。输出私钥$SK=(F_1,F_2;h_1,h_2;P_1,P_2)$，公钥$PK=(g_1,g_2;A_0,\ldots,A_k;B_0,\ldots,B_k;D_0,\ldots,D_k;H,H')$。

- BuildIndex算法：数据所有者首先选择一个随机数$r_1\in Z_q$，计算$u_1=g_1^{r_1}$，$u_2=g_2^{r_1}$。然后对于关键词w，依次计算$A_w=\prod_{t=0}^{k}A_t^{w^t}$，$B_w=\prod_{t=0}^{k}B_t^{w^t}$，$D_w=\prod_{t=0}^{k}D_t^{w^t}$，$s=D_w^{r_1}$，$e=(0^k)\otimes H'(s)$，$a=H(u_1,u_2,e)$，$v_w=(A_w)^{r_1}(B_w)^{r_1a}$。最后输出关键词w对应的密文索引$C=(u_1,u_2,e,v_w)$；
- GenTrapdoor算法：对于检索关键词w，数据检索者使用私钥生成陷门$T_w=<F_1(w),F_2(w),h_1(w),h_2(w),P_1(w),P_2(w)>$。
- Search算法：给定某关键词的密文索引C和查询关键词w的陷门T_w，服务器计算$a=H(u_1,u_2,e)$，并测试v_w是否等于$(u_1)^{F_1(w)+h_1(w)a}\cdot(u_2)^{F_2(w)+h_2(w)a}$，如果不等于则停止，否则计算$s=(u_1)^{P_1(w)}\cdot(u_2)^{P_2(w)}$，$m=e\otimes H'(s)$。如果计算结果为$0^k$，那么C是查询关键词w的一个加密文字



DS-PEKS方案

DS-PEKS方案主要基于雅可比符号以及二次剩余中二次不可区分性问题。与前两小节的构造思路类似，该方案同样为各关键词分别生成索引。记$Z_n^{+1}(Z_n^{-1})$ 表示一个正整数集合，且其元素满足三个条件：①小于n；②与n互素；③雅克比符号等于+1(-1)。给定安全参数m和关键词长度k，以及哈希函数$H:\{0,1\}^k \to Z_n^{+1}$，DS-PEKS方案的具体步骤如下：

1. Setup算法：数据检索者随机选取两个长度为m/2的素数p和q，且这两个素数都符合3mod4，令n=pq。输出公钥pk=(n,1^k)以及私钥sk=(p,q)；

2. BuildIndex算法：当数据所有者为关键词w生成索引时，对于每一个i=1,...,4k：

2.1）计算h_i=H(w|i)，并随机选取$u_i \in Z_n^*$；

2.2）如果$\left(\frac{u_i^2-4h_i}{n}\right)=+1$，则随机选取$t_i \in Z_n^{+1}$，并计算$s_i=\left(t_i+\frac{h_i}{t_i}\right) mod\ n$；

2.3）如果$\left(\frac{u_i^2-4h_i}{n}\right) \in \{-1,0\}$，则$s_i=u_i$；最后，输出索引$s=(s_1,\dots,s_{4k})$；

3. GenTrapdoor算法：假设数据检索者希望为关键词W生成陷门，对于每一个i=1,...,4k：

3.1）计算h_i=H(W|i)；

3.2）使用p和q随机选择$g_i \in Z_n^*$，且$g_i^2 = h_i mod\ n$；如果g_i不存在，则记$g_i = \perp$。

最后，输出陷门$g=(g_1,\dots,g_{4k})$；

4. Search算法：给定索引s和陷门g，对于每一个i=1,...,4k：

4.1）如果$g_i = \perp$，则$\bar{t}_i = \perp$。

4.2）如果$g_i^2 = h_i mod\ n$，那么再考虑s_i：如果$\left(\frac{s_i^2-4h_i}{n}\right)=+1$，则$\bar{t}_i=(\frac{s_i+2g_i}{n})$，否则，$\bar{t}_i = \perp$。最后，如果对于所有i，均满足$\bar{t}_i \in \{+1,\perp\}$，则匹配成功，否则，匹配失败。



方案对比

非对称密文检索方案对比

非对称密文检索方案	通信量	服务器检索效率	加密效率
BDOP-PEKS	|g|	1次对运算	1次对运算
KR-PEKS	6|g|	O(1)	O(k)
DS-PEKS	4|g|logN	O(N)	O(N)

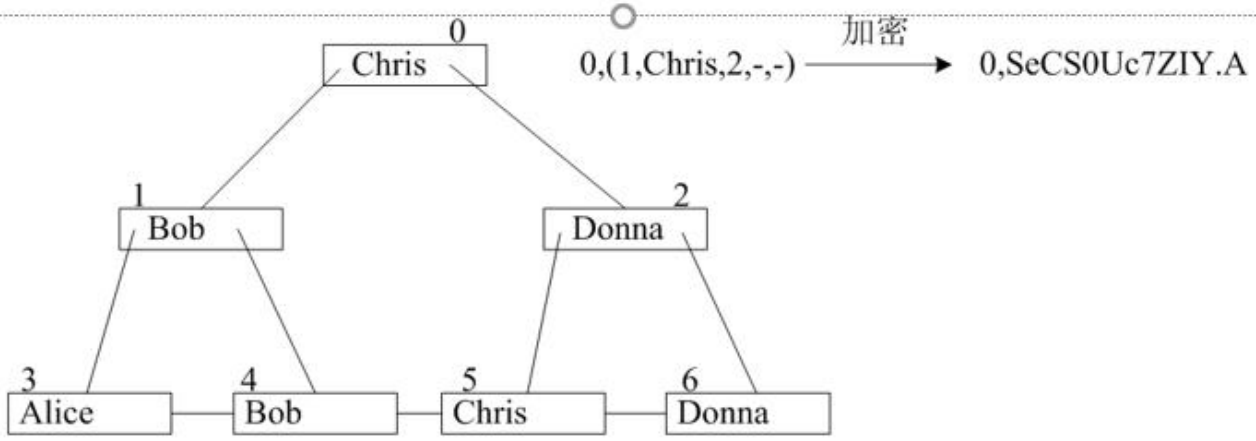
注：|g|表示群中元素所占用的存储空间，N表示字典中的关键词个数，k为安全参数



密文区间检索

基于B+树和传统加密技术的方案

- 主要思路：为数据构造B+树，并使用传统加密技术将各节点分别加密后存储到服务器。当需要进行检索时，服务器将密文节点返回给用户，由用户在解密后进行判断，然后通知服务器下一个需要查询的密文节点。
- 方案特点：客户端工作量较大，且需要服务器和客户端之间进行多轮交互操作。





基于保序加密的方案

保序加密（Order-Preserving Encryption，OPE）是一种保持数值顺序关系的加密算法，其算法本身可以应用于各种场景，并不仅限于区间检索。

定义3-13 （保序加密） 给定加密算法E，如果对于任何密钥k，以及数值数据x和y（x<y），均存在$E_k(x)<E_k(y)$，则称E是保序的。

由于本身的保序特性，保序加密很难保证数据的隐私性。Durak等人指出，即使是达到IND-OCPA安全性的保序加密算法，也无法抵御具备特定背景知识的敌手的攻击。



延伸阅读

延伸阅读：

（1）Ref. 23: Searchable Symmetric Encryption: improved definition and efficient construction.

（2）QQ群里的search encryption：survey。